Two-Stage Calibration of NCDawareRank — Technical Documentation

Ioannis Konstantellos
Computer Engineering and Informatics Department, University of Patras

October 2026

Contents

1. Purpose of this document **2**
1.1 The research question in one paragraph . . . 2
1.2 Headline results (all numbers are reproduced by the code) . . . 2

2. Repository layout **2**

3. Software architecture **3**
3.1 Data flow . . . 3
3.2 Design principles . . . 4

4. Environment **4**

5. Module-by-module walkthrough **5**
5.1 Notation shared by all modules . . . 5
5.2 `ncd_calibration_lib.py` (core library) . . . 5
5.3 `decode_cnr2000.py` and `prepare_real_graphs.py` . . . 6
5.4 `run_experiments.py` (driver) . . . 7
5.5 Stage 1 analysis scripts . . . 7
5.6 Stage 2 analysis scripts . . . 8
5.7 `generate_figures.py` and `fig_lang.py` . . . 8
5.8 `make_paper_assets.py` . . . 9
5.9 Unit tests . . . 9

6. Complexity and solver mechanics **9**
6.1 Building the operators in $O(m+n)$ time and space . . . 9
6.2 The calibration statistics in $O(m+n)$ (Proposition 5 of the paper) . . . 9
6.3 The power iteration . . . 10
6.4 The Neumann-series α-sweep . . . 10
6.5 Why exact ties matter . . . 10
6.6 Other costs . . . 10

7. Ground truths **11**

8. Panels, seeds and perturbations **11**

9. Statistical methodology **12**

10. Outputs: files, figures and tables **12**
10.1 Result files . . . 12
10.2 Figures . . . 13
10.3 Tables of the paper . . . 13

11. Step-by-step reproduction **13**

12. Limitations to keep in mind when citing the results **14**

13. Mapping to thesis chapters **14**

1. Purpose of this document

This guide documents the software, data, experiments and outputs behind the manuscript *Two-Stage Calibration of Decomposable Web Ranking: Block Escape Probability, Incoming-Transition Deviation, and the Anchoring of Oracle Damping* (`paper/paper_en.pdf`, Greek version `paper/paper_gr.pdf`). It is written so that each part can be lifted into a diploma thesis: Section 3 describes the architecture, Section 5 walks through every module, Section 6 proves the complexity claims and explains the Neumann-series solver, Sections 7–9 define the ground truths, panels and statistics, Section 10 explains every output file, figure and table, and Section 11 gives step-by-step reproduction instructions.

1.1 The research question in one paragraph

NCDawareRank (Nikolakopoulos and Garofalakis, WSDM 2013, cited as [NG13]) ranks Web pages with the Markov chain $P = \eta H + \mu M + (1-\eta-\mu)E$, where H is the hyperlink matrix, M the inter-level proximity matrix defined over NCD blocks (for example, Web sites) and $E = ev^\top$ a block-even teleportation matrix. Writing $\eta = (1-r)\alpha$ and $\mu = r\alpha$ separates the *mixing ratio* r from the *total damping* α. The study tests a zero-cost, two-stage calibration:

- **Stage 1** fixes $\alpha_0 = 0.85$ and chooses $r = f(\bar{P}_{\text{esc}}^{(M)})$ from the mean block escape probability $\bar{P}_{\text{esc}}^{(M)} = \frac{1}{n}\sum_u k_u/(k_u+1)$, where k_u is the number of distinct external blocks page u links to.
- **Stage 2** keeps that r and chooses $\alpha = g(h)$ from the mean absolute deviation $h = \frac{1}{n}\sum_v |1-\Sigma_{\text{in}}(v;r)|$ of the column sums of $Z_r = (1-r)H + rM$ from one.

1.2 Headline results (all numbers are reproduced by the code)

Item	Result
Locked Stage 1 rule	$r = \text{clip}(0.242 - 0.143\,\bar{P},\ 0.01,\ 0.99)$; best constant $r = 0.18$
Locked Stage 2 rule	the τ-optimised rule on h collapses to the constant $\alpha = 0.810$ (0.930 under NCD95)
Two-stage vs. canonical $(r,\alpha) = (0.118, 0.85)$	`test` $+0.0110$ Kendall τ (31/32 graphs, $p = 9.3 \times 10^{-10}$); `scale` $+0.0102$; `topo` $+0.0036$; `real` $+0.0017$ ($p = 0.812$, n.s.)
Predictive value of $\bar{P}$, h, h_v	none out of sample; h falls under link removal and rises under the newly-added-pages perturbation, while α^* falls under both
Non-monotone and dangling-deconfounded g	none of 127 deployable rules (tents, quadratic, Gaussian bump, steps, splines on h, h_v, h_{raw}, δ, ζ; bivariate $g(h,\delta)$, $g(h,\zeta)$) passes the pre-registered held-out test; the τ-optimised tent on h changes τ by -0.0001 out of sample; with the graph's own unperturbed h_0 (not observable) a tent in $h - h_0$ gains $+0.0018$ in CV and $+0.0006$ out of sample, so $\alpha = 0.810$ stays locked
Anchoring	oracle α^* equals the damping of the reference ranking minus a 0.03–0.06 noise discount; dose-response slope 0.950
Theory	$h = \|u^\top Z_r - u^\top\|_1$, $h_v = \|v^\top Z_r - v^\top\|_1 = \|\mathrm{d}\pi/\mathrm{d}\alpha\|_{\alpha=0}$, and $\|\pi(\alpha) - v\|_1 \le \frac{\alpha}{1-\alpha} h_v$
Convergence	lowering α $0.85 \to 0.81$ saves 14.7% of power iterations on `test` (a-priori estimate 22.9%)

2. Repository layout

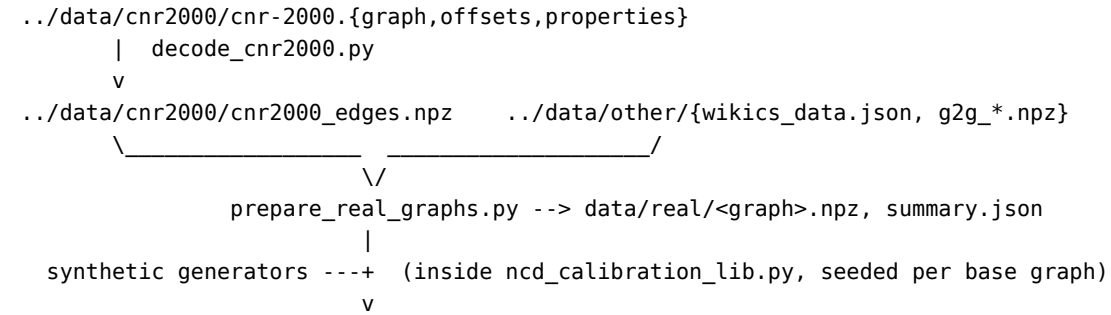
```
ncdcal/
  ncd_calibration_lib.py    core library: operators, solvers, statistics,
                            perturbations, generators, rank metrics, Stage 2 sweeps
```

```
decode_cnr2000.py         BVGraph files of cnr-2000 -> plain arc list
prepare_real_graphs.py    builds data/real/*.npz and data/real/summary.json
run_experiments.py        experiment driver: stage1 | stage2 (multiprocessing)
run_stage1_all.sh         runs every Stage 1 panel in sequence
run_stage2_all.sh         runs every Stage 2 panel in sequence
stage1_analysis.py        oracle r*, candidate rules f, evaluation, diagnostics
stage1_decomposition.py   variance decomposition of r*
stage1_feature_probe.py   diagnostic: block-informativeness statistics
stage1_crosstruth.py      rules fitted under one truth, evaluated under others
stage1_tables.py          Checkpoint-1 markdown tables
stage2_analysis.py        oracle alpha*, candidate rules g, pipelines, diagnostics
stage2_decomposition.py   variance decomposition and anchors of alpha*
stage2_rapr_anchor.py     dose-response test with Random-alpha references
stage2_upper_check.py     censoring check at the top of the alpha grid (truth Q)
stage2_tables.py          Checkpoint-2 markdown tables
stage2_nonmonotone_search.py
                          non-monotone and dangling-deconfounded rules g(h), g(h, delta),
                          g(h, zeta) against the locked constant (Section 8.2 of the paper)
build_papers.sh           regenerates assets and compiles both manuscripts
build_docs.sh             builds the PDF versions of both documentation files
verify_stage2.py          independent recomputation of the headline numbers
convergence_check.py      power-iteration counts before/after the noise discount
bound_check.py            numerical check of the displacement bound
generate_figures.py       the six publication figures (EN and GR)
fig_lang.py               Greek translation layer for the figures
make_paper_assets.py      numbers (macros) and tables of both manuscripts
tests/test_lib.py         unit tests of the library (Stage 1 parts)
tests/test_stage2.py      unit tests of the Stage 2 additions
tests/test_nonmono.py     unit tests of stage2_nonmonotone_search.py
data/real/                prepared real graphs (npz) and summary.json
results/stage1/           raw per-graph files and Stage 1 analysis outputs
results/stage2/           raw per-graph files and Stage 2 analysis outputs
results/paper_numbers.json  every number quoted in the manuscripts
figures/, figures_gr/     publication figures (PDF + PNG), English / Greek
logs/                     console logs of the runs
paper/                    paper_en.tex/.pdf, paper_gr.tex/.pdf, generated/
DOCUMENTATION_EN.md/.pdf, DOCUMENTATION_GR.md/.pdf
METHOD_DISCLOSURE_EN_GR.md/.txt   method disclosure (EN and GR), kept outside the manuscripts
```

The raw downloads (BVGraph files of cnr-2000, the WikiCS JSON and the CitationFull archives) live one level up, in `../data/`, because they are inputs rather than products of the pipeline.

3. Software architecture

3.1 Data flow



```
run_experiments.py stage1 --> results/stage1/<panel>/<base>.npz
     stage1_analysis.py --> instances.csv, fits_<GT>.json, summary_<GT>.csv,
                            diagnostics_<GT>.json
     stage1_decomposition.py, stage1_crosstruth.py, stage1_feature_probe.py,
     stage1_tables.py
                        |  (Stage 1 rules locked at Checkpoint 1)
                        v
run_experiments.py stage2 --> results/stage2/<panel>/<base>.npz
     stage2_analysis.py --> instances.csv, fits.json, summary.csv,
                            pipelines.csv, diagnostics.json
     stage2_decomposition.py, stage2_rapr_anchor.py, stage2_upper_check.py,
     convergence_check.py, bound_check.py, verify_stage2.py, stage2_tables.py
     stage2_nonmonotone_search.py --> nonmono_features.csv, nonmono_rules.csv,
                                      nonmono_summary.json
                        v
generate_figures.py (+ fig_lang.py) --> figures/, figures_gr/
make_paper_assets.py --> paper/generated/{macros.tex, en/, gr/},
                         results/paper_numbers.json
xelatex paper_en.tex / paper_gr.tex --> paper_en.pdf / paper_gr.pdf
```

3.2 Design principles

1. **One library, thin scripts.** All mathematics lives in `ncd_calibration_lib.py`. The scripts orchestrate runs, fit rules and write outputs; they contain no independent implementation of the chain (the single exception is `verify_stage2.py`, which deliberately re-derives the headline numbers from the raw files without importing the analysis code).
2. **Determinism.** Every base graph has its own seed (Section 8). Its perturbations are drawn from the same `numpy.random.Generator`, in a fixed order, so the Stage 2 driver and the diagnostic scripts *replay* exactly the instances of Stage 1. `stage1_feature_probe.py` checks the replay against the stored rows.
3. **The base graph is the unit of replication.** Perturbed instances of one base graph share its structure. Every paired test and every bootstrap interval is computed over per-graph means, and every split (calibration/test/transfer, cross-validation folds) is by base graph.
4. **Caching and restartability.** Each base graph produces one compressed `npz` file. The driver skips files that already exist, so an interrupted run resumes where it stopped.
5. **No hand-typed numbers.** Every number in both manuscripts is a LaTeX macro and every table is generated, both by `make_paper_assets.py` from the result files.
6. **Exact rank statistics.** Solvers use only sparse matrix–vector products and entry-independent scalings, so structurally identical pages keep bit-identical scores and Kendall's τ sees the true ties (Section 6.5).

4. Environment

Component	Version used
Python	3.13
NumPy / SciPy	2.5.3 / 1.18.1
pandas	3.0.5
scikit-learn	1.9.1 (gradient boosting, ridge, isotonic regression, cross-validation)
matplotlib	3.11.2
webgraph (Python bindings of webgraph-rs)	0.2.0, only for `decode_cnr2000.py`
XeLaTeX	TeX Live, with `fontspec`, `txfonts`, `babel` (English, Greek), FreeSerif font
pandoc	for the PDF versions of this documentation

Install the Python dependencies with `pip install numpy scipy pandas scikit-learn matplotlib webgraph`. The experiments ran with two worker processes; Stage 1 took about 69 minutes and Stage 2 about 101 minutes of wall time on the full panel set, dominated by the largest real graph (cnr-2000).

5. Module-by-module walkthrough

5.1 Notation shared by all modules

Symbol	Code	Meaning
n, N, m	`ops.n`, `ops.N`, `ops.m`	pages, blocks, arcs
$\mathcal{G}_u, d_u$	`src`, `dst`, `ops.d_out`	out-neighbours and out-degree
$\mathcal{A}_{(u)}$	`ops.block[u]`	block of page u
$N_u = k_u + 1$	`ops.k_ext + 1`	blocks in the proximal set $\mathcal{X}_u$
$H^\top$	`ops.HT` (CSR, $n \times n$)	transposed hyperlink matrix without dangling rows
$R^\top$	`ops.RT` (CSR, $N \times n$)	transposed block-proximity factor, $M = RA$
A	`ops.block` (index map)	page-to-block incidence, applied by indexing
v	`ops.v`	block-even teleportation, $v_u = 1/(N\lvert\mathcal{A}_{(u)}\rvert)$
r, α	`r`, `alpha`	mixing ratio and total damping, $\eta = (1-r)\alpha$, $\mu = r\alpha$

Dangling pages ($d_u = 0$) have uniform rows $e^\top/n$ in H. They are not stored: every product with H adds the rank-one term $(\sum_{u \text{ dangling}} x_u)/n$.

5.2 `ncd_calibration_lib.py` (core library)

Constants. `ALPHA0 = 0.85`; `R_CANONICAL = 0.10/0.85` (the canonical split $(\eta, \mu) = (0.75, 0.10)$ of [NG13, Table 2]); `R_GRID = 0.00, 0.01, …, 0.99` (Stage 1 grid); `ALPHA_GRID` = 0.05, 0.10, …, 0.35 followed by 0.40, 0.41, …, 0.99 (67 values; the oracle uses only the values ≥ 0.40, `ALPHA_ORACLE_MASK`); `ALPHA_NG13 = 0.95`, `R_NG13 = 0.10/0.95` (the setting of the ranking experiments of [NG13, §4]).

1. Graph container. `Graph(n, src, dst, block, name, meta)` holds a directed graph with an NCD partition; `subgraph_arcs(keep)` returns the same pages and partition with a subset of the arcs. `clean_arcs(n, src, dst)` removes self-loops and duplicate arcs by sorting integer keys $u \cdot n + v$.

2. Operators. `build_operators(g)` returns `NCDOperators` with fields `n, N, m, block, block_size, d_out, dangling, HT, RT, k_ext, n_ext_arcs, v`:

- `HT` is built directly in transposed form, $H^\top_{vu} = 1/d_u$ for every arc $u \to v$;
- the non-zeros of R are the distinct pairs (page, block) formed by each page's own block and the blocks of its out-neighbours; `N_u` counts them, and $R^\top_{Iu} = 1/(N_u\lvert\mathcal{A}_I\rvert)$;
- $v_u = 1/(N\,\lvert\mathcal{A}_{(u)}\rvert)$.

The implementation finds the distinct pairs with one vectorised `np.unique` on integer keys (an $O(m' \log m')$ sort, $m' = n+$ inter-block arcs). Proposition 5 of the paper gives the $O(\lvert\mathcal{E}\rvert + n)$ marker-array construction of the same pairs (Section 6.1).

3. Solver. `ncdawarerank(ops, alpha, r, x0=None, tol=1e-10, maxit=20000, teleport=None)` is the factorised power iteration of [NG13, Algorithm 1]:

$$\pi^{(k+1)} = \eta\,\pi^{(k)\top} H + \mu\,(\pi^{(k)\top} R)A + (1-\alpha)v^\top,$$

started at v (or at `x0`), renormalised to sum one, stopped when $\|\pi^{(k+1)} - \pi^{(k)}\|_1 <$ `tol`; it returns `(pi, iterations)`. `sweep_r(ops, alpha, r_values)` solves along a grid of r, warm-starting each solve from the previous solution (Stage 1). `dense_P(ops, eta, mu)` builds the dense matrices for the unit tests only.

4. Statistics.

- `block_escape(ops)` returns $P^{(M)}_{\text{esc}}(u) = k_u/(k_u+1)$ and its mean $\bar{P}$.
- `incoming_sums(ops)` returns $\Sigma^{(H)}_{\text{in}}(v) = \sum_{u\to v} 1/d_u + n_{\text{d}}/n$ (row sums of `HT` plus the dangling term) and $\Sigma^{(M)}_{\text{in}}(v)$ (row sums of `RT`, copied from blocks to pages).

- `h_statistic(ops, r)` returns $h(r)$ and the mean of Σ_{in} (which must equal 1).
- `h_statistics(ops, r)` returns `h`, `mean_sigma` and `h_v`$= \|v^\top Z_r - v^\top\|_1$ (one product with $Z_r^\top$).
- `structural_features(ops)` returns every observable global statistic used by the study: `pesc_M` ($\bar{P}$), `pesc_M_linked` (mean over non-dangling pages), `mean_k`, `pesc_H_block` (block escape under H: share of a page's out-arcs that leave its block; $1 - |\mathcal{A}_{(u)}|/n$ for dangling pages), `inter_arc_frac`, `mean_outdeg`, `log_mean_outdeg`, `dangling_frac`, `zero_in_frac` (pages without in-links), `n`, `N`, `m`, `mean_block_size`, `log_n`.

5. Perturbations ([NG13, §4.2]). `sparsify_arcs(g, keep_frac, rng)` keeps a uniformly random fraction of the arcs (Experiment A). `newly_added_pages(g, page_frac, rng, remove=0.9)` chooses a fraction of the pages and deletes each of their incoming arcs with probability 0.9 (Experiment B).

6. Synthetic generators.

- `sample_static_params(rng, n)` draws the static-generator parameters: mean block size log-uniform in $[4, 100]$, block-size dispersion, domain size (3–30 blocks), domain locality $\lambda \in [0, 0.9]$, latent-quality dispersion $\sigma_q \in [0.8, 2]$, block share of quality (ICC) $\in [0, 0.8]$, block attractiveness exponent $\beta \in [0.5, 1.2]$ and noise, mean out-degree log-uniform in $[2, 25]$, out-degree dispersion, dangling share $\in [0.02, 0.30]$ and intra-block probability $\varphi \in [0.30, 0.97]$.
- `generate_static_web(params, rng)` builds the graph and returns it together with a `PopulationModel`: the generator's *expected* walk, available in closed form. Its method `pagerank(alpha, v)` computes the population truth POP and its field `q` the latent quality Q.
- `sample_growth_params` / `generate_growth_web` build the growth model used for topology transfer: pages arrive one at a time, found a new site (probability 0.01–0.25) or join an existing one in proportion to its size, link by preferential attachment on in-degree (offset 0.2–5) inside their site with probability φ, and an update phase adds 10–50% more arcs.

7. Rank metrics. `kendall_tau_b(x, y)` (SciPy's τ-b); `KendallRef(y)` computes τ-b against a fixed reference with the reference sorted once ($O(n \log n)$ per call, identical to SciPy, verified by a unit test); `SpearmanRef`, `spearman_rho`, `centered_ranks` and `spearman_from_ranks` compute Spearman's ρ with shared rank computations.

8. Rule helpers. `clip_linear(x, c0, c1, lo, hi)`, `logistic(x, a, b, lo, hi)` and `interp_curve(curves, r_hat)` (linear interpolation of stored $\tau(r)$ curves at a predicted r).

9. Stage 2 machinery. `zt_multi(ops, X, r)` computes $Z_r^\top X$ for a block of vectors with a different r per column. `alpha_sweep(ops, r_values, alphas, tail=1e-13, block=32)` returns $\pi(\alpha; r)$ for all 67 values of α and all requested r from one Neumann series per r (Section 6.4). `rapr_reference(ops, r, a, b, K)` computes Random-α NCDawareRank $\mathbb{E}[\pi(A; r)]$, $A \sim$ Beta(a, b), with weights $w_k = B(a + k, b + 1)/B(a, b)$; $a = b = 1$ is TotalRank, $w_k = 1/((k + 1)(k + 2))$.

5.3 `decode_cnr2000.py` and `prepare_real_graphs.py`

`decode_cnr2000.py [raw_dir]` opens the BVGraph files of cnr-2000 (Laboratory for Web Algorithmics) with `webgraph.BvGraph`, reads every successor list and writes `cnr2000_edges.npz` (`src`, `dst`, `n`; 325,557 pages and 3,216,152 arcs, checked against the `.properties` file). It reproduces the arc list used in the experiments exactly.

`prepare_real_graphs.py [names…]` writes `data/real/<name>.npz` (`n`, `src`, `dst`, `block`) and `data/real/summary.json` (sizes, block statistics, intra-block arc share, dangling share, mean out-degree) for five graphs:

Name	Source file	Blocks
`cnr2000`	`../data/cnr2000/cnr2000_edges.npz`	host proxy: contiguity-constrained greedy maximisation of directed modularity over the URL-ordered page ids (7,604 segments)
`wikics`	`../data/other/wikics_data.json` (WikiCS, Mernyei & Cangea 2020)	10 topic labels
`cora_full`	`../data/other/g2g_cora.npz` (CitationFull, Bojchevski & Günnemann 2018)	70 topic labels
`cora_ml`	`../data/other/g2g_cora_ml.npz`	7 topic labels
`citeseer`	`../data/other/g2g_citeseer.npz`	6 topic labels

Self-loops and duplicate arcs are removed. The host proxy is needed because the URL list of cnr-2000 is not distributed; LAW

numbers pages in lexicographic URL order, so every host is an interval of ids, and `contiguous_modularity_segmentation` starts from singletons and repeatedly merges the adjacent pair of segments with the largest positive directed-modularity gain

$$\Delta Q(a,b) = \frac{e_{ab} + e_{ba}}{m} - \frac{\text{out}_a \,\text{in}_b + \text{out}_b \,\text{in}_a}{m^2},$$

where $e_{ab} + e_{ba}$ counts the arcs between the two segments (a priority queue holds the candidate merges, with lazy invalidation of stale entries); pages without arcs join the segment on their left. The topic-label partitions follow the "category of its content" criterion that [NG13, §5] lists as an admissible decomposition.

5.4 `run_experiments.py` (driver)

```
python3 run_experiments.py stage1 --panel calib [--workers 2]
python3 run_experiments.py stage2 --panel test,topo --workers 2
python3 run_experiments.py stage1 --panel all
```

`panel_specs(panel)` lists the base graphs of a panel with their seeds (Section 8); `make_base(spec)` generates or loads the base graph. Jobs run in a process pool, largest graphs first.

Stage 1 (`run_base`). For each base graph: build the operators; compute the references NCD and PR (tolerance 10^{-12}, 10^{-11} for graphs above 10^5 pages), POP for static graphs, and the full-graph rankings for every r on `R_GRID` (needed by SELF). Then, for every perturbation level and replicate, rebuild the operators on the observed graph, compute `structural_features`, rank with NCDawareRank$(\alpha_0 = 0.85, r)$ for all 100 values of r (warm-started, tolerance 10^{-10}), and store Kendall's τ-b and Spearman's ρ against every truth, plus the values at the canonical r. Output `results/stage1/<panel>/<base>.npz` with arrays `tau`, `rho` (instances × truths × r), `tau_can`, `rho_can`, `iters` and a JSON `meta` (spec, generator parameters, rows of features, grids, runtime).

Stage 2 (`run_base_stage2`). Replays the same instances. References: NCD, NCD95, TOT (`rapr_reference` with $K = 20{,}000$ terms, 10,000 for large graphs), and POP and Q for static graphs. For each instance it evaluates the six r-settings of `R_NAMES` (Table below), computes h, h_v and the mean incoming sum for each, plus h and h_v at $r = 0$ and $r = 1$, runs `alpha_sweep` for all active r at once, and stores τ and ρ against the truths listed in `PAIRS`. Output `results/stage2/<panel>/<base>.npz` with `tau`, `rho` (instances × r-settings × truths × 67 α), `r`, `h`, `h_v`, `mean_sigma` and `meta`.

r-setting	Value	Evaluated against
`can`	$0.10/0.85 = 0.118$	all truths
`ng13`	$0.10/0.95 = 0.105$	all truths
`const`	0.18 (best Stage 1 constant)	NCD, TOT, POP, Q
`rule`	clip$(0.242 - 0.143\bar{P}, 0.01, 0.99)$ (locked)	all truths
`popLS`	clip$(0.420 - 0.386\bar{P}, 0.01, 0.99)$	POP, Q (static only)
`popTAU`	clip$(0.448 - 0.547\bar{P}, 0.01, 0.99)$	POP, Q (static only)

`run_stage1_all.sh` and `run_stage2_all.sh` run all panels in sequence.

5.5 Stage 1 analysis scripts

- **`stage1_analysis.py [--gt NCD|PR|POP|SELF|all].`** Loads the raw files, builds `instances.csv` (one row per instance: features, generator parameters, and for every truth the Kendall- and Spearman-optimal r^*, τ^*, τ at the canonical r and at $r = 0$, and the flatness of the curve). Fits the candidate rules on `calib` (constant; clipped-linear by least squares on r^* and by maximising mean τ; free-bound variants; logistic; exponential; isotonic; diagnostic multi-feature rules and gradient-boosted trees; competitor features such as `pesc_H_block`, `mean_k`, `inter_arc_frac`, `dangling_frac`). Evaluates every rule on every panel (mean τ and ρ, share of the oracle, gap closed relative to the canonical r, paired tests against the canonical r and against the constant), runs grouped 8-fold cross-validation, and writes `fits_<GT>.json`, `summary_<GT>.csv` and `diagnostics_<GT>.json` (noise ceiling, within/between decomposition, Kendall-vs-Spearman agreement, share of oracles at the grid bounds).
- **`stage1_decomposition.py`.** Variance decomposition of r^* for every truth (graph identity, perturbation level, both; boosted trees on hidden generator parameters; ridge on observables; $\bar{P}$ alone) and correlations of graph-mean r^* with each hidden parameter (`decomposition.json`).

- **`stage1_feature_probe.py`.** A pre-declared diagnostic of three block-informativeness statistics (`corr_HM_log`, `icc_indeg`, `disp_H`) on replayed instances (`probe_features.csv`).
- **`stage1_crosstruth.py`.** Evaluates rules fitted under one truth against every other truth (`crosstruth.csv`) and breaks the real panel down by graph (`real_by_graph.csv`).
- **`stage1_tables.py`.** Writes the Checkpoint-1 tables (`checkpoint1_tables.md`).

5.6 Stage 2 analysis scripts

- **`stage2_analysis.py [--pipelines-only]`.** Loads the Stage 2 files, computes the oracle α^* (argmax of τ over $\alpha \in [0.40, 0.99]$) and its unrestricted counterpart, and for each pair (r-setting, truth, feature $\in \{h, h_v\}$) fits the candidate forms of g: constant, clipped-linear (least squares and τ-optimised), free bounds, logistic, isotonic, displacement budget $c/(c+x)$, and boosted trees on all observables (diagnostic ceiling). Every rule is clipped to $[0.40, 0.99]$ and evaluated by linear interpolation of the stored $\tau(\alpha)$ curves on the non-uniform grid. It writes `instances.csv`, `fits.json`, `summary.csv` (rule × panel: mean τ, differences against $\alpha = 0.85$ and against the constant with Wilcoxon p, gap closed, CV τ), `diagnostics.json` (within/between correlations, α^* distribution, the check max $|\overline{\Sigma}_{\text{in}} - 1|$) and `pipelines.csv` (complete (r, α) pipelines against both static settings for every truth and panel).
- **`stage2_decomposition.py`.** Variance decomposition of α^* (with h alone as the feature of interest) and the anchors α^* on the unperturbed graph (`decomposition.json`).
- **`stage2_rapr_anchor.py [n_graphs=16]`.** Dose-response test: Random-α references with priors Beta(3,7), Beta(5,5), Beta(1,1), Beta(7,3), Beta(8.5,1.5) (means 0.30, 0.50, 0.50, 0.70, 0.85); writes `rapr_anchor.csv`.
- **`stage2_upper_check.py [n_graphs=8]`.** Evaluates τ against Q at $\alpha \in \{0.99, 0.995, 0.999\}$ (`upper_check.csv`).
- **`convergence_check.py [panels]`.** Power-iteration counts at tolerance 10^{-6} for $\alpha = 0.85$ vs 0.81 (rule r) and 0.95 vs 0.93 ($r = 0.105$) (`convergence.csv`).
- **`bound_check.py [n_calib=4]`.** Ratio of the two sides of the displacement bound on the unit-test graphs and on calibration graphs (`bound_check.csv`); asserts that the ratio never exceeds one.
- **`verify_stage2.py`.** Recomputes the pipeline numbers per panel directly from the raw files (no analysis code imported) and prints them with the Wilcoxon test of the two-stage pipeline.
- **`stage2_tables.py`.** Writes the Checkpoint-2 tables (`checkpoint2_tables.md`).
- **`stage2_nonmonotone_search.py [--replay] [--workers 2]`.** Tests whether the inverted-V shape of α^* in h (falling under link removal, rising under newly added pages, α^* falling under both) can be exploited. It reuses the cached surfaces $\tau(\alpha)$ of every instance (no sweep is rerun; τ at a predicted α is interpolated) and replays the perturbed graphs with the seeds and random stream of `run_experiments.py stage2` only to obtain per-page column sums (about 15 s), asserting that the replayed h, r and dangling fraction equal the stored ones (maximum deviation 2.2×10^{-16}). Statistics: h, h_v; the dangling-deconfounded deviation $h_{\text{raw}} = \text{mean}_v|1 - \Sigma_{\text{in}}(v) + (1-r)\delta|$, its relative version $h^{\text{rel}}_{\text{raw}}$ and the raw hyperlink-matrix deviation h^{H}_{raw}; the dangling fraction δ and the share ζ of pages without in-links. Families: clipped-linear, asymmetric and symmetric tent, concave quadratic, Gaussian bump, exact two- and three-region step functions, two-knot linear spline (each by least squares and by direct τ-maximisation: grid, then Nelder–Mead and Powell; apex of rules on h in $[0.50, 0.70]$), bivariate clipped-linear, tent+$c\,s$, 3×3 cells and boosted trees on (h, δ) and (h, ζ), plus the non-deployable diagnostics $h - h_0$ (own unperturbed value) and the per-graph best constant. Exact 8-fold CV by graph with the constant refitted per fold; paired Wilcoxon and bootstrap over per-graph means; Holm across the four held-out panels; pooled held-out test over 73 graphs. The decision rule (gain ≥ 0.0005 with $p < 0.05$ in CV and pooled held-out, no Holm-significant loss) is fixed in the docstring. Configurations: NCD with the Stage 1 rule r (primary), NCD95 with $r = 0.105$ and TOT (sensitivity). Writes `nonmono_features.csv`, `nonmono_rules.csv` and `nonmono_summary.json`; about 8 minutes with two workers.

5.7 `generate_figures.py` and `fig_lang.py`

```
python3 generate_figures.py stage1 [--only NAME] [--lang en|gr]
python3 generate_figures.py stage2 [--only NAME] [--lang en|gr]
```

The script writes PDF (for LaTeX) and PNG (for previews) to `figures/` or, with `--lang gr`, to `figures_gr/`. `fig_lang.patch()` wraps matplotlib's text entry points (titles, axis labels, tick labels, free text, legend labels, colour-bar labels) so that every string passes through an ordered list of literal and regular-expression replacements, longest first; mathematics in `$…$` is left untouched and panel letters (a)–(d) become (α)–(δ). The figures use a validated three-colour categorical palette (blue, orange, aqua), hairline grids and legends placed outside the data.

5.8 `make_paper_assets.py`

Reads every result file and writes `paper/generated/macros.tex` (about 300 `\newcommand` macros, each mode-agnostic: signed numbers and powers of ten are wrapped in `\ensuremath`), `paper/generated/en/tab_*.tex` and `paper/generated/gr/tab_*.tex` (the same tables with English or Greek captions and labels) and `results/paper_numbers.json`. Both manuscripts `\input` these files, so a re-run of the analysis updates every number and table of both papers consistently.

5.9 Unit tests

`python3 tests/test_lib.py` checks: the matrices M, R, A of the tiny graph of [NG13, Fig. 1] entry by entry; row-stochasticity of H, M and P; the factorised power iteration against a dense solve; Theorem 1 of [NG13] ($|\lambda_2(P)| \le \eta + \mu$) and Theorem 2 (block-level stationary vector); the block escape probability; the incoming sums and their unit mean; the exact form of [NG13, Remark 1] (one block gives $M = ee^\top/n$; singleton blocks give the lazy walk $(D+I)^{-1}(\mathrm{Adj}+I)$, not H); the population model; and the perturbations.

`python3 tests/test_stage2.py` checks: `KendallRef` against SciPy with ties; `alpha_sweep` against the power iteration for every α (to 10^{-11} in L_1); exact tie preservation; TotalRank against Gauss–Legendre quadrature of dense solves; the identities for h and $\overline{\Sigma}_{\text{in}} = 1$ and h_v against a finite-difference derivative; and the displacement bound for every α of the grid.

`python3 tests/test_nonmono.py` checks: the vectorised evaluation of mean τ over many parameter vectors against row-by-row interpolation; that the exact step-function fits attain the brute-force optimum (mean τ and least squares); that every non-monotone family reduces to the constant when its slopes vanish and that the tent is unimodal with a clipped apex; the identity between the deconfounded column sums and the column sums of $(1-r)H_{\text{raw}} + rM$ (mean $1-(1-r)n_{\text{d}}/n$); and that the replay of one calibration graph reproduces its stored h, r and dangling fraction.

6. Complexity and solver mechanics

Throughout, $m = |\mathcal{E}|$ is the number of arcs, n the number of pages and N the number of blocks; $\mathrm{nnz}(H) = m$ and $\mathrm{nnz}(R) \le n + m$.

6.1 Building the operators in $O(m+n)$ time and space

Claim. $H^\top$, the non-zeros of R, the block map A and v can be built in $O(m+n)$ time with $O(m+n)$ memory.

Proof. Out-degrees are one counting pass over the arcs, $O(m+n)$. $H^\top$ in CSR form is a counting sort of the arcs by target, $O(m+n)$, with value $1/d_u$ per arc. For R, keep an array $\mathrm{last}[1..N]$ initialised to zero; for each page u visit its own block and the block of each out-neighbour, and record the pair (u, I) only when $\mathrm{last}[I] \neq u$, then set $\mathrm{last}[I] \leftarrow u$. Each page costs $O(1+d_u)$, so all pairs are found in $O(\sum_u (1+d_u)) = O(n+m)$ time; their count per page is N_u, and the values $1/(N_u|\mathcal{A}_I|)$ need the block sizes, one $O(n)$ pass. v is $O(n)$. Memory: m entries for $H^\top$, at most $n+m$ for $R^\top$, $n+N$ for the block map and sizes, n for v. □

Implementation note. The library obtains the same pairs with one vectorised `np.unique` call, which sorts $m' = n +$ (inter-block arcs) integer keys, i.e. $O(m' \log m')$ time. For the graph sizes of this study the sort is a negligible part of the run time; the marker-array version above is the one that attains the stated bound.

6.2 The calibration statistics in $O(m+n)$ (Proposition 5 of the paper)

Claim. $\{k_u\}$, $\bar{P}^{(M)}_{\text{esc}}$, $\Sigma^{(H)}_{\text{in}}$, $\Sigma^{(M)}_{\text{in}}$, $h(r)$ and $h_v(r)$ cost $O(m+n)$ time and $O(n+N)$ memory beyond the graph.

Proof. $k_u = N_u - 1$ comes with the construction of R; $\bar{P}$ is one $O(n)$ pass. $\Sigma^{(H)}_{\text{in}}(v) = \sum_{u \to v} 1/d_u + n_{\text{d}}/n$ is one pass over the arcs plus an $O(n)$ addition. $\Sigma^{(M)}_{\text{in}}$ accumulates $1/N_u$ into each proximal block of each page, $O(\mathrm{nnz}(R))$, divides by block sizes and copies to pages, $O(n)$. $h(r)$ is one $O(n)$ pass. $h_v(r) = \|v^\top Z_r - v^\top\|_1$ needs $v^\top Z_r = (1-r)v^\top H + r(v^\top R)A$, that is, one sparse product with $H^\top$ ($O(m+n)$ including the dangling term), one with $R^\top$ ($O(\mathrm{nnz}(R))$) and one gather ($O(n)$). The extra memory is the marker array and block sums ($O(N)$) and a constant number of page vectors ($O(n)$). □

Because both statistics are by-products of building R, the two-stage procedure adds no asymptotic cost to NCDawareRank.

6.3 The power iteration

One iteration costs $O(\mathrm{nnz}(H) + \mathrm{nnz}(R) + n)$: a sparse product with $H^\top$, the dangling correction, a product with $R^\top$ that produces N block masses, a gather to pages, and the teleportation term.

Convergence. With $x_k = \pi^{(k)\top}$, the teleportation term cancels in successive differences: $x_{k+1} - x_k = \alpha(x_k - x_{k-1})Z_r$. A row-stochastic matrix does not increase the L_1 norm of a row vector, so $\|x_{k+1} - x_k\|_1 \le \alpha^k \|x_1 - x_0\|_1 \le 2\alpha^k$. The stopping rule $\|x_{k+1} - x_k\|_1 < \varepsilon$ is therefore met after at most $\lceil \ln(\varepsilon/2)/\ln\alpha \rceil + 1$ iterations, and the total cost is $O((m+n)\log(1/\varepsilon)/\log(1/\alpha))$. In practice the contraction is faster (the subdominant eigenvalue is $\alpha\lambda_2(Z_r)$ with $|\lambda_2(Z_r)| \le 1$, [NG13, Theorem 1]), which is why the measured iteration counts in `convergence.csv` are well below this estimate. Memory: three page vectors in addition to the operators.

6.4 The Neumann-series α-sweep

For fixed r the ranking has the series form

$$\pi(\alpha)^\top = (1-\alpha)\sum_{k\ge 0} \alpha^k u_k, \qquad u_k = v^\top Z_r^k,$$

in which only the scalar weights depend on α. `alpha_sweep` therefore computes the iterates $u_0 = v^\top$, $u_{k+1} = u_k Z_r$ once and accumulates every α_j of the grid from them:

$$\hat\pi(\alpha_j)^\top = (1-\alpha_j)\sum_{k=0}^{K_j} \alpha_j^k u_k + \alpha_j^{K_j+1} u_{K_j+1}, \qquad K_j = \lceil \ln(\text{tail})/\ln\alpha_j \rceil,\ \text{tail} = 10^{-13}.$$

Exactness. The weights sum to one, $\sum_{k\le K}(1-\alpha)\alpha^k + \alpha^{K+1} = 1$, so $\hat\pi$ is a probability vector. The error is $\pi - \hat\pi = \sum_{k>K}(1-\alpha)\alpha^k u_k - \alpha^{K+1}u_{K+1}$; both terms are probability vectors scaled by $\sum_{k>K}(1-\alpha)\alpha^k = \alpha^{K+1} \le \text{tail}$, hence $\|\pi - \hat\pi\|_1 \le 2\,\text{tail} = 2\times 10^{-13}$.

Cost. The largest truncation index belongs to $\alpha = 0.99$: $K_{\max} = \lceil \ln 10^{-13}/\ln 0.99 \rceil = 2{,}979$ (for $\alpha = 0.40$ it is only 33). The sweep performs $K_{\max}+1$ products with $Z_r^\top$, batched over all requested r values (one column per r), i.e. $O(K_{\max}\,|r|\,(m+n))$, plus the accumulation. Iterates are buffered in blocks of $B = 32$ (16 for large graphs) and folded into the 67 outputs with one dense matrix product (GEMM) per block and per r, using a weight matrix that already contains the $(1-\alpha_j)\alpha_j^k$ factors and the tail correction; outputs whose series has ended are skipped. The accumulation costs $O(K_{\max}\cdot n_\alpha \cdot n)$ flops per r but runs at BLAS speed. Memory: $O(|r|\,n_\alpha\,n)$ for the outputs and $O(|r|\,B\,n)$ for the buffer. Compared with 67 separate power iterations, the sweep shares every sparse product among all values of α.

Agreement. The unit tests confirm agreement with the power iteration to 10^{-11} in L_1 for every α on the grid, and the Stage 2 replay reproduces the Stage 1 values of τ at $\alpha = 0.85$ to 10^{-6}.

6.5 Why exact ties matter

Pages with identical in-neighbourhoods in the same block receive identical scores in exact arithmetic. The power iteration and the Neumann sweep apply the same sequence of floating-point operations to such pages (the same CSR rows, the same block mass, the same teleportation weight), so their scores stay bit-identical, which the unit test `test_alpha_sweep_ties` verifies. A pivoted direct solver mixes rows and breaks these ties with rounding noise. Kendall's τ-b then counts spurious concordant or discordant pairs; on the 2,000-page graphs this lowered τ by 0.002–0.004, a sizeable fraction of the effects studied. All solvers of the package therefore use only products and entry-independent scalings.

6.6 Other costs

- `rapr_reference`: $K+1$ products with $Z_r^\top$, $O(K(m+n))$; the probability mass beyond K terms ($1/(K+2)$ for TotalRank) is assigned to the last iterate.
- `KendallRef`: $O(n \log n)$ per evaluation (one stable sort and a merge-sort discordance count).
- Per Stage 2 instance: one sweep per active r-setting (batched) and $n_\alpha \times$ (truths) rank correlations.

7. Ground truths

All references are computed on the **complete** base graph (except Q, which is latent) with block-even teleportation and uniform dangling rows. A method is scored by Kendall's τ-b between its ranking of the **observed** (perturbed) graph and the reference.

Truth	Definition	Panels	Role
NCD	NCDawareRank, $(\alpha, r) = (0.85, 0.118)$, i.e. $(\eta, \mu) = (0.75, 0.10)$	all	primary truth of both stages
NCD95	NCDawareRank, $(\alpha, r) = (0.95, 0.105)$, i.e. $(\eta, \mu) = (0.85, 0.10)$, the setting of the ranking experiments of [NG13]	all (Stage 2)	sensitivity: anchor at 0.95
PR	PageRank ($r = 0$), $\alpha = 0.85$	all (Stage 1)	methodological contrast: favours $r \to 0$
SELF	the candidate's own ranking on the complete graph with the same (α, r), the stability protocol of [NG13]	all (Stage 1)	shown to be degenerate: rewards $r \to 1$ and $\alpha \to 0$
POP	PageRank at 0.85 of the static generator's expected walk	static panels	population importance not defined by any method
Q	latent quality q of the static generator	static panels (Stage 2)	walk-free reference
TOT	$\int_0^1 \pi(\alpha)\,\mathrm{d}\alpha = \sum_k v^\top Z_r^k/((k+1)(k+2))$ with $r = 0.118$: TotalRank, i.e. Random-α NCDawareRank with a Beta(1,1) prior (mean 0.5)	all (Stage 2)	α-integrated reference
RAPr family	$\mathbb{E}[\pi(A)]$, $A \sim \mathrm{Beta}(a, b)$, means 0.30–0.85	16 calibration graphs	dose-response test of anchoring

Why several truths? The study found that the optimal damping is set by the reference itself: on the unperturbed graph, α^* is 0.852 (NCD), 0.950 (NCD95), 0.547 (TOT), 0.767 (POP) and 0.971 (Q). Reporting one truth would turn a property of the reference into an apparent property of the method.

8. Panels, seeds and perturbations

Panel	Generator	Base graphs	Seeds	n	Replicates	Instances (Stage 2)	Role
`calib`	static fitness block web	64	1000–1063	2,000	2	1,856	fitting
`test`	static, independent seeds	32	5000–5031	2,000	1	480	held-out
`scale`	static, 8× larger	12	9000–9011	16,000	1	180	scale transfer
`topo`	growth / preferential attachment	24	13000–13023	3,000	1	360	topology transfer
`real`	Table 1 of the paper	5	17000–17004	2,995–325,557	2 (1 for cnr-2000)	131	real-network transfer

Every base graph yields the unperturbed instance A1.0, Experiment A (keep 90%, 80%, …, 20% of the arcs) and Experiment B (10%, …, 60% of the pages lose each in-arc with probability 0.9), each level with the stated number of replicates: $1 + 14 \times$

replicates instances per base graph, 3,007 instances in total. A1.0 is excluded from evaluations under NCD, NCD95 and TOT, where it is a trivial recovery problem.

9. Statistical methodology

- **Oracles.** $r^* = \arg\max_{r\in[0.01,0.99]} \tau$ at $\alpha_0 = 0.85$; $\alpha^* = \arg\max_{\alpha\in[0.40,0.99]} \tau$ with r locked.
- **Rules** are fitted on `calib` only; τ-optimised fits maximise the mean calibration τ over the stored curves (decision-focused fitting), least-squares fits regress the oracle values.
- **Model selection** uses 8-fold cross-validation grouped by base graph.
- **Paired comparisons** use per-graph mean differences with the Wilcoxon signed-rank test and 20,000-resample percentile bootstrap intervals over graphs; the number of graphs on which a policy wins is reported. Panel-level means in the tables weight instances; paired tests weight graphs, which matters only on the real panel (per-graph $+0.0017$ vs per-instance $+0.0000$ for the two-stage pipeline).
- **Gap closed** is $(\tau_{\text{rule}} - \tau_{\text{static}})/(\tau_{\text{oracle}} - \tau_{\text{static}})$.
- **Within/between decomposition**: correlations of de-meaned values inside graphs and of graph means across graphs (Simpson structure).
- **Variance decomposition**: in-sample ANOVA shares of graph identity and perturbation level; out-of-sample R^2 (grouped CV) of boosted trees on hidden generator parameters, ridge regression on all observable statistics, and the clip-linear rule on the statistic alone.

10. Outputs: files, figures and tables

10.1 Result files

File	Content
`results/stage1/<panel>/<base>.npz`	raw Stage 1 curves $\tau(r)$, $\rho(r)$ per instance and truth; iterations; metadata
`results/stage1/instances.csv`	one row per instance: features, generator parameters, r^*, τ^*, canonical values per truth
`results/stage1/fits_<GT>.json`	every fitted Stage 1 rule (form, parameters, formula)
`results/stage1/summary_<GT>.csv`	rule × panel: τ, ρ, oracle share, gap closed, tests vs canonical and constant, CV τ
`results/stage1/diagnostics_<GT>.json`	noise ceiling, within/between, bounds, Kendall–Spearman agreement, CV
`results/stage1/decomposition.json`	variance decomposition of r^* per truth
`results/stage1/crosstruth.csv`, `real_by_graph.csv`	cross-truth evaluation, per-graph real results
`results/stage2/<panel>/<base>.npz`	raw Stage 2 curves $\tau(\alpha)$, $\rho(\alpha)$ per instance, r-setting and truth; r, h, h_v
`results/stage2/instances.csv`	per instance: features, r, h, h_v per r-setting, α^*, τ^*, τ at 0.85/0.95 per pair
`results/stage2/fits.json`	fitted Stage 2 rules per (r-setting, truth, feature)
`results/stage2/summary.csv`	rule × panel results of Stage 2
`results/stage2/pipelines.csv`	complete pipelines vs both static settings, every truth and panel
`results/stage2/diagnostics.json`	correlations, α^* statistics, CV, numerical checks
`results/stage2/decomposition.json`	variance decomposition and anchors of α^*
`results/stage2/rapr_anchor.csv`	dose-response: α^* per instance and prior
`results/stage2/upper_check.csv`	τ against Q at 0.99, 0.995, 0.999
`results/stage2/convergence.csv`	iteration counts per instance
`results/stage2/bound_check.csv`	both sides of the displacement bound per (graph, r, α)
`results/stage2/nonmono_features.csv`	replayed per-instance statistics: h re-derived, h_{raw}, $h^{\text{rel}}_{\text{raw}}$, h^{H}_{raw}, δ
`results/stage2/nonmono_rules.csv`	one row per (configuration, rule): parameters, calib in-sample, CV and held-out comparisons with the constant, decision flags
`results/stage2/nonmono_summary.json`	trajectories along both experiments, directions of movement, dispersion of h_0, decision
`results/paper_numbers.json`	every number quoted in the manuscripts

10.2 Figures

1. **`stage1_calibration_scatter.`** Oracle r^* against $\bar{P}$ on `calib`. Panel (a): truth NCD with the fitted rules and the canonical r; panel (c): truth POP; panels (b) and (d): the same points with each graph's perturbation series joined and graph means marked, showing the steep within-graph slope and the flat between-graph slope (Simpson structure).
2. **`stage1_gt_sensitivity.`** r^* against $\bar{P}$ under NCD, PR, SELF and POP: PR pushes r^* to the lower bound, SELF to the upper bound.
3. **`stage1_diagnostics.`** Panel (a): $\tau(r)$ curves of one graph under increasing link loss; panel (b): variance decomposition of r^* (NCD, PR, POP); panel (c): graph-mean r^* (POP) against the latent ICC, coloured by quality dispersion; panel (d): out-of-sample gap closed by the rule and by the constant.
4. **`stage2_calibration_scatter.`** Oracle α^* against h at the Stage 1 r. Panel (a): NCD with the fitted candidates of g (least squares, τ-optimised $= 0.810$, isotonic, budget); panel (b): perturbation series joined; panels (c) and (d): TOT and Q. Horizontal lines mark 0.85 and 0.95.
5. **`stage2_gt_sensitivity.`** α^* against h under the five truths, each with its own damping as a horizontal line and the within/between correlations.
6. **`stage2_diagnostics.`** Panel (a): $\tau(\alpha) - \max \tau$ for one graph (Experiment A, 60% of arcs kept) under each truth; panel (b): distribution of α^* per truth with each truth's own α; panel (c): variance decomposition of α^*; panel (d): gain of complete pipelines over the static setting $(0.118, 0.85)$ on `test` under NCD, TOT and Q.

10.3 Tables of the paper

Table	Content	Generator function
1 `tab:real`	real networks (size, blocks, intra-block arcs, dangling share, $\bar{P}$)	`real_dataset_table`
2 `tab:panels`	panels, generators, roles, graphs, instances, sizes	`panel_table`
3 `tab:stage1`	Stage 1 candidate rules: CV τ, τ per panel, gap closed, p	`stage1`
4 `tab:stage1dec`	variance decomposition of r^* per truth	`stage1`
5 `tab:stage2`	Stage 2 candidate rules on h and h_v, differences vs the locked constant	`stage2`
6 `tab:stage2dec`	variance decomposition of α^* per truth and r-setting	`stage2`
7 `tab:nonmono`	non-monotone, dangling-deconfounded and bivariate Stage 2 rules vs the locked constant	`nonmono`
8 `tab:anchor`	anchors: own α, α^* on A1.0, mean α^*, discount, within/between	`stage2`
9 `tab:rapr`	dose-response: α^* per Random-α prior	`stage2`
10 `tab:pipe`	complete pipelines vs static settings, five truths × five panels	`pipelines`
11 `tab:conv`	power-iteration counts and savings	`convergence`
12 `tab:stage1truth`	Stage 1 under the four Stage 1 truths (appendix)	`stage1_truth_table`
13 `tab:sens`	Stage 2 rules refitted per sensitivity truth (appendix)	`stage2`
14 `tab:realdetail`	real panel per network (appendix)	`real_detail`

11. Step-by-step reproduction

All commands run from the `ncdcal/` directory.

1. **Install** Python 3.13 and `pip install numpy scipy pandas scikit-learn matplotlib webgraph`; for the manuscripts, a TeX distribution with XeLaTeX and the FreeSerif font; for this documentation, pandoc.
2. **Unit tests** (a few seconds): `python3 tests/test_lib.py && python3 tests/test_stage2.py && python3 tests/test_nonmono.py`.

3. **Raw data** into `../data/`: the cnr-2000 BVGraph files (`cnr-2000.graph`, `.offsets`, `.properties`) from the Laboratory for Web Algorithmics (https://law.di.unimi.it/webdata/cnr-2000/) into `../data/cnr2000/`; the WikiCS file `dataset/data.json` of https://github.com/pmernyei/wiki-cs-dataset (saved as `wikics_data.json`) and the CitationFull archives `cora.npz`, `cora_ml.npz`, `citeseer.npz` of the graph2gauss repository (https://github.com/abojchevski/graph2gauss, saved as `g2g_<name>.npz`) into `../data/other/`.
4. **Decode cnr-2000**: `python3 decode_cnr2000.py`.
5. **Prepare the real graphs**: `python3 prepare_real_graphs.py` (writes `data/real/`).
6. **Stage 1 experiments**: `bash run_stage1_all.sh` (about 70 minutes with two workers).
7. **Stage 1 analysis**: `python3 stage1_analysis.py`, `python3 stage1_decomposition.py`, `python3 stage1_crosstruth.py`, `python3 stage1_tables.py` (optionally `python3 stage1_feature_probe.py`).
8. **Stage 2 experiments**: `bash run_stage2_all.sh` (about 100 minutes).
9. **Stage 2 analysis**: `python3 stage2_analysis.py`, `python3 stage2_decomposition.py`, `python3 stage2_rapr_anchor.py`, `python3 stage2_upper_check.py`, `python3 convergence_check.py`, `python3 bound_check.py`, `python3 verify_stage2.py`, `python3 stage2_tables.py`, and `python3 stage2_nonmonotone_search.py` (about 8 minutes).
10. **Figures**: `python3 generate_figures.py stage1`, `python3 generate_figures.py stage2`, and the same two commands with `--lang gr`.
11. **Manuscripts**: `bash build_papers.sh` runs `make_paper_assets.py` and compiles `paper/paper_en.tex` and `paper/paper_gr.tex` with two XeLaTeX passes each.
12. **Documentation PDFs**: `bash build_docs.sh` converts both Markdown guides with pandoc and XeLaTeX (FreeSerif text, Latin Modern Math, `-V fontfamily=fontspec` so that the default template does not request the `lmodern` package).

Checks. `verify_stage2.py` must reproduce the mean τ of every pipeline in Table 10 and the Wilcoxon tests of the two-stage pipeline (its printed mean differences weight instances, so on the real panel it shows $+0.0000$ where Table 10, which weights graphs equally, shows $+0.0017$); `bound_check.py` asserts the displacement bound; `stage2_analysis.py` prints the maximum deviation of the mean incoming sum from one (3.3×10^{-16}); `stage2_nonmonotone_search.py` asserts that the replayed h, r and dangling fraction equal the stored ones (maximum deviation 2.2×10^{-16}).

12. Limitations to keep in mind when citing the results

- The synthetic generators are the author's; transfer to other generators is not guaranteed.
- The cnr-2000 partition is a host proxy, and the citation graphs use topic labels, not hosts.
- Dangling pages use uniform rows, not the block-confined handling proposed in the extended NCDawareRank work.
- Stage 1 is anchored at $\alpha_0 = 0.85$; the Stage 2 grid stops at 0.99, which censors the oracle under Q (the upper check shows the optimum does not lie beyond it).
- Only rank correlations against model-derived references are evaluated; external relevance judgements would provide a damping-free ground truth.
- The real-panel gain of the two-stage pipeline is not significant ($p = 0.812$, 2 of 5 graphs).

13. Mapping to thesis chapters

Thesis chapter	Paper sections	Code and outputs
Background: PageRank, NCD, NCDawareRank	1–2	`ncd_calibration_lib.py` §1–3
Stage 1 statistic	3	`block_escape`, `structural_features`
Stage 2 statistic and theory	4	`incoming_sums`, `h_statistics`, `bound_check.py`, `tests/test_stage2.py`
Algorithm and complexity	5	Section 6 of this guide
Experimental design	6	`run_experiments.py`, `prepare_real_graphs.py`, generators
Stage 1 results	7	`stage1_*.py`, Figures 1–3, Tables 3, 4, 12
Stage 2 results	8	`stage2_*.py`, Figures 4–6, Tables 5–11, 13, 14

Thesis chapter	Paper sections	Code and outputs
Discussion and conclusions	9–10	—